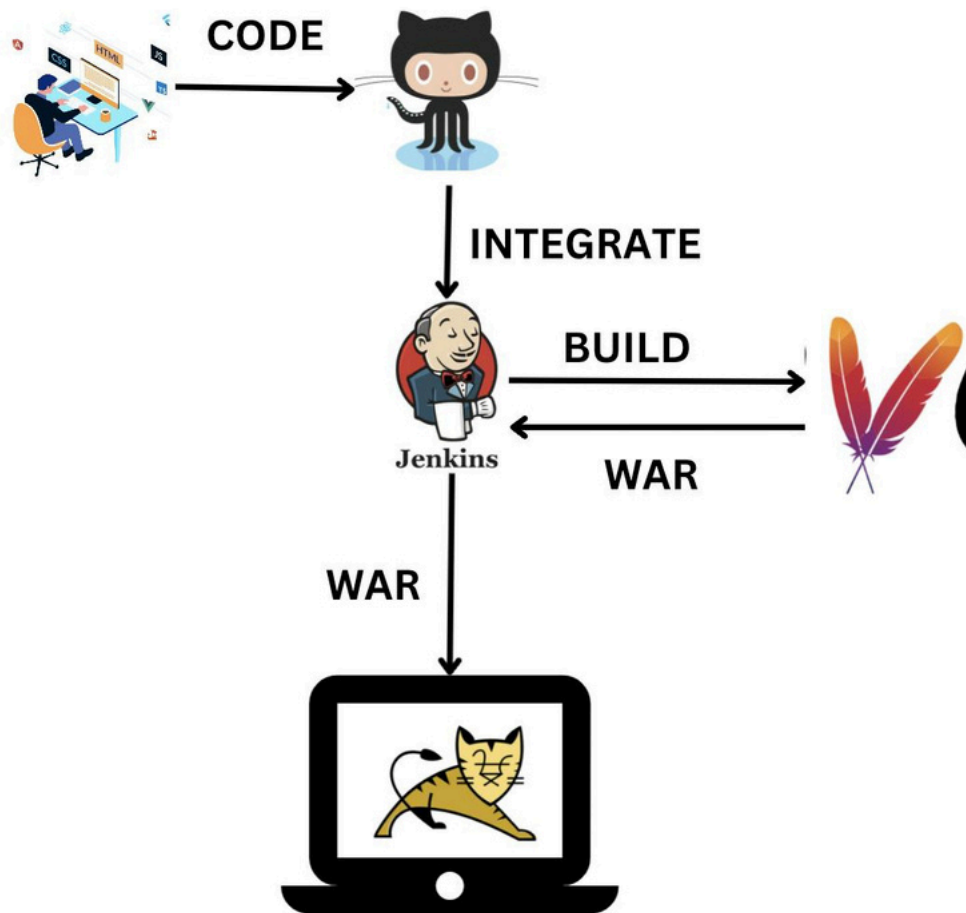


Deploy a Java Application using Jenkins



[Shaik Mustafa](#)

• [Dec 25, 2024](#) •

4 min read

Introduction

In today's fast-paced software development world, Continuous Integration and Continuous Deployment (CI/CD) streamline application delivery. In this blog, we'll discuss how to deploy a Java-based application to an Apache Tomcat server using a CI/CD pipeline. We'll use Jenkins for automation, Maven for build management, and Git for version control.

Understanding the Tools

1. **Git:** Stores your application code and tracks changes.

2. **Maven:** Builds and manages Java projects by handling dependencies.
3. **Jenkins:** Automates the CI/CD process by integrating code building, testing, and deployment.
4. **Tomcat:** A popular server for hosting Java-based applications.

Instances (2) Info

Last updated less than a minute ago

Connect

Instance state

Actions

Launch instances

Find Instance by attribute or tag (case-sensitive)

All states

< 1 >

☐	Name	Instance ID	Instance state		Instance type	Status check	Alarm status
☐	Jenkins	i-0b950cb28d3b5d0c9	Running		t2.micro	Initializing	View alarms +
☐	Tomcat	i-0aa48f381b89f683b	Running		t2.micro	Initializing	View alarms +

Real-Time Scenario

Imagine a software team working on an e-commerce web application. Every time a developer updates the product catalog, they push the changes to GitHub. Jenkins automatically:

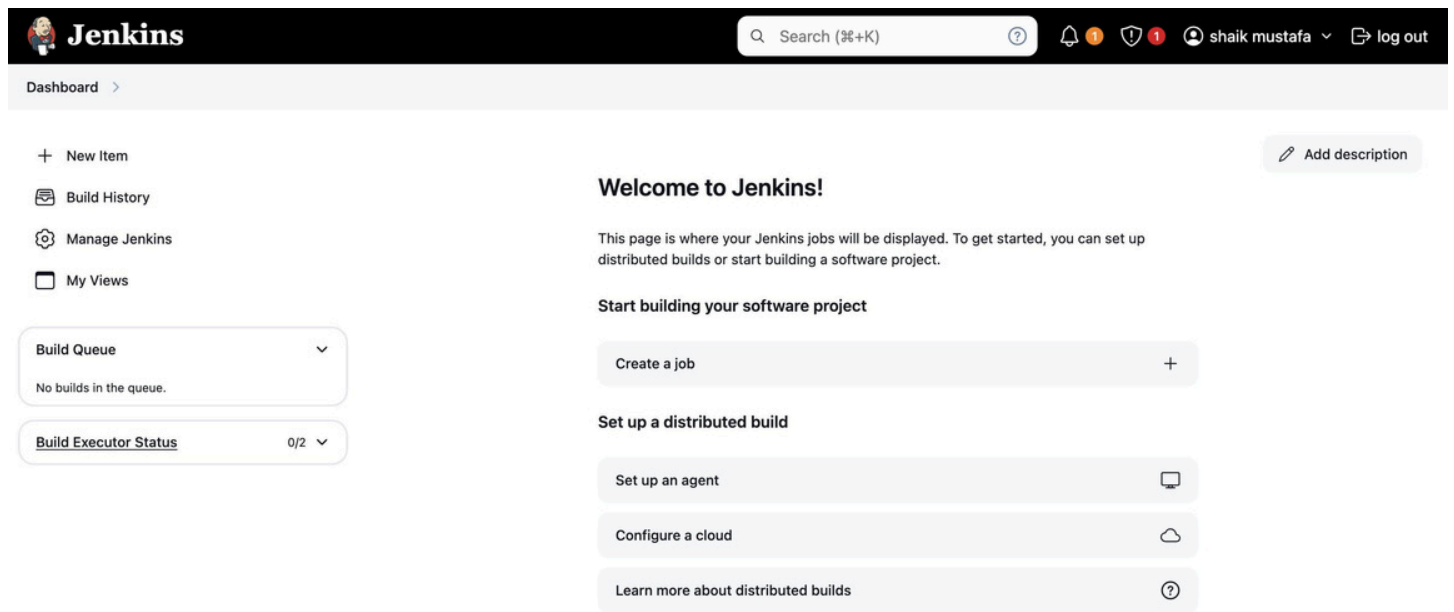
1. Pulls the updated code.
2. Builds the application using Maven.
3. Perform the unit test using Maven.
4. Deploys the new version to Tomcat.

This process minimizes manual effort and ensures the application is always up-to-date.

Steps to Deploy a Java-Based Application Using CI/CD

STEP-1: Launch 2 EC2 Instances (Kernel 5.10)

- Jenkins Server
- Tomcat Server

The image shows the Jenkins web interface. At the top is a black header with the Jenkins logo, a search bar, and user information. Below the header is a light gray sidebar with navigation links: 'New Item', 'Build History', 'Manage Jenkins', and 'My Views'. The main content area has a 'Welcome to Jenkins!' message and instructions on how to get started. It includes two sections: 'Start building your software project' with a 'Create a job' button, and 'Set up a distributed build' with buttons for 'Set up an agent', 'Configure a cloud', and 'Learn more about distributed builds'. On the left, there are two status boxes: 'Build Queue' (empty) and 'Build Executor Status' (0/2).

STEP-2: Connect Jenkins server and Install Jenkins & GIT

Copy

Copy

#STEP-1: INSTALLING GIT

```
yum install git -y
```

#STEP-2: GETTING THE REPO (jenkins.io --> download --> redhat)

```
sudo wget -O /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/redhat-stable/jenkins.repo
```

```
sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2023.key
```

#STEP-3: DOWNLOAD JAVA17 AND JENKINS

```
yum install java-17-amazon-corretto -y
```

```
yum install jenkins -y
```

#STEP-4: RESTARTING JENKINS (when we download service it will on stopped state)

`systemctl start jenkins.service`

`systemctl status jenkins.service`

Now copy the **public-ip** of your system and access it with **8080** port.

Dashboard > Manage Jenkins > Plugins

Plugins

Updates

Available plugins

Installed plugins

Advanced settings

Download progress

Deploy to container

Install

Deploy to container 1.16

Artifact Uploaders

This plugin allows you to deploy a war to a container after a successful build.
Glassfish 3.x remote deployment

Released

4 yr 1 mo ago

STEP-3: Now Install **Deploy to container** plugin

Go to Manage Jenkins » Plugins » Available Plugins » Deploy to container

Dashboard > Manage Jenkins > Tools

Maven installations

Add Maven

Maven

Name

mymaven

☒ Install automatically ?

Install from Apache

Version

3.9.9

Add Installer

Add Maven

Save

Apply

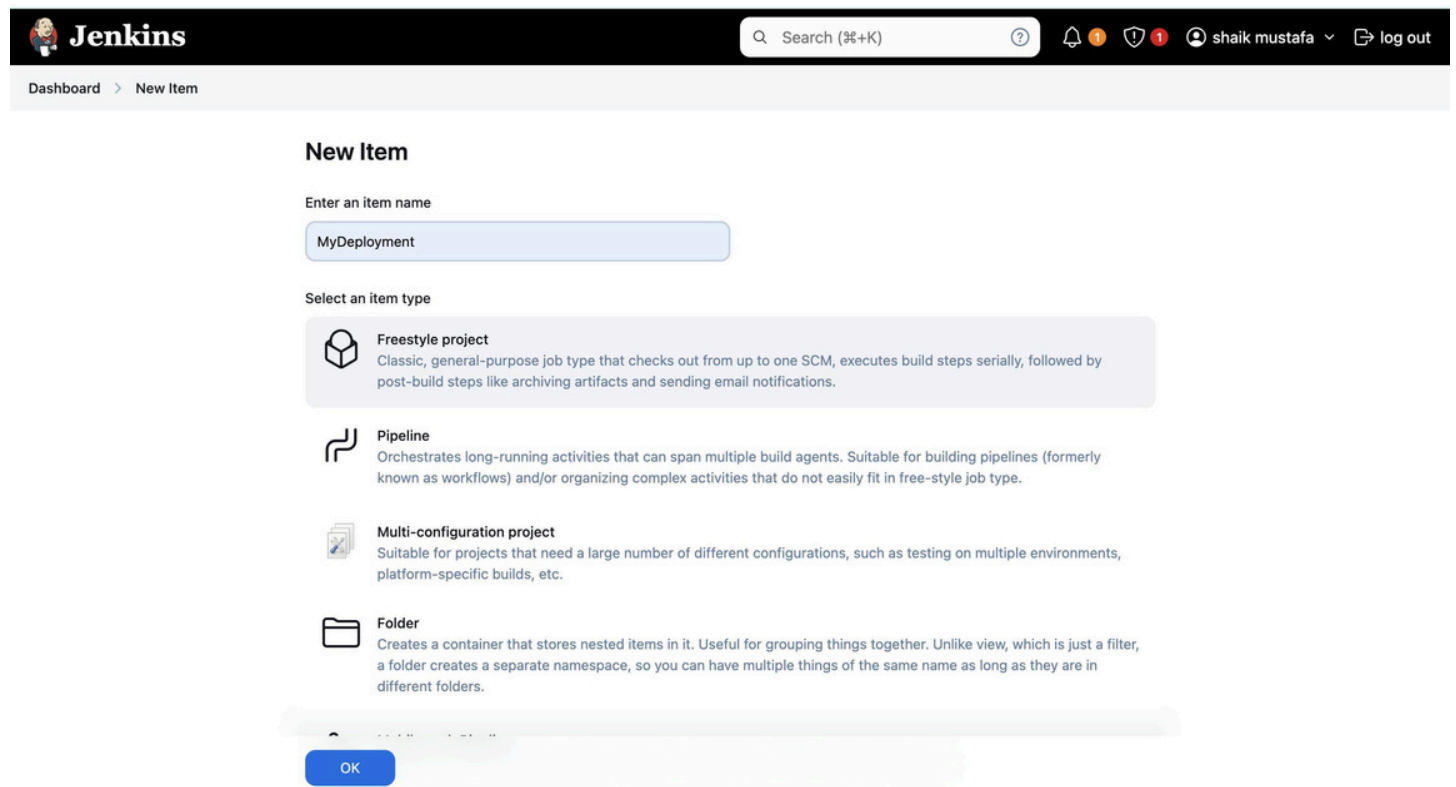
STEP-4: Integrate Maven

Go to Manage Jenkins » tools and select maven

under the **Maven Installations** click on **Add Maven**

Name: **mymaven**

Version: **default (3.9.9)**



Jenkins Search (⌘+K) ? shaik mustafa log out

Dashboard > New Item

New Item

Enter an item name

MyDeployment

Select an item type

-  **Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
-  **Pipeline**
Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.
-  **Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
-  **Folder**
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

OK

Now click on **save**

STEP-5: Create a Free Style Job

Dashboard > MyDeployment > Configuration

Configure

- General
- Source Code Management
- Build Triggers
- Build Environment
- Build Steps
- Post-build Actions

☐ Invoke builds ?

☐ Execute concurrent builds if necessary ?

Advanced ▾

Source Code Management

☐ None

☒ Git ?

Repositories ?

Repository URL ?

<https://github.com/usubbu/one.git>

Credentials ?

- none - ▾

+ Add

Advanced ▾

Go to **Source Code Management** and select **Git** and enter the repo-url
(<https://github.com/usubbu/one.git>)

Dashboard > MyDeployment > Configuration

Configure

- General
- Source Code Management
- Build Triggers
- Build Environment
- Build Steps
- Post-build Actions

☐ Delete workspace before build starts

☐ Use secret text(s) or file(s) ?

☐ Add timestamps to the Console Output

☐ Inspect build log for published build scans

☐ Terminate a build if it's stuck

☐ With Ant ?

Build Steps

≡ **Invoke top-level Maven targets** ?

Maven Version

mymaven ▾

Goals

clean package ▾

Advanced ▾

Add build step ▾

Save Apply

Go to **Build Steps** and select **Invoke top-level Maven targets** and select the Maven version as **mymaven**

Now enter the goal as **clean package**

Dashboard > MyDeployment >

Status

</> Changes

Workspace

Build Now

Configure

Delete Project

Rename

Builds

⋮

Today

✓

#1 18:29

▼

MyDeployment

Permalinks

Save the job and click on **Build Now**

- Status
- Changes
- Workspace
- Build Now
- Configure
- Delete Project
- Rename

Workspace of MyDeployment on Built-In Node

MyDeployment / target /

maven-archiver

myweb-8.6.7

myweb-8.6.7.war 24 Dec 2024, 18:29:58 1.96 KiB

(all files in zip)

Builds

Filter

Today

#1 18:29

If the build gets success, go to **workspace** and open the **target** folder, we will get war file

← → ↻ Not Secure 54.88.54.240:8080 ☆

Home Documentation Configuration Examples Wiki Mailing Lists Find Help

Apache Tomcat/9.0.98

If you're seeing this, you've successfully installed Tomcat. Congratulations!

 Recommended Reading:

- [Security Considerations How-To](#)
- [Manager Application How-To](#)
- [Clustering/Session Replication How-To](#)

Server Status
Manager App
Host Manager

Developer Quick Start

- [Tomcat Setup](#)
- [Realms & AAA](#)
- [Examples](#)
- [Servlet Specifications](#)
- [First Web Application](#)
- [JDBC DataSources](#)
- [Tomcat Versions](#)

Managing Tomcat

For security, access to the `manager webapp` is restricted. Users are defined in:

```
$CATALINA_HOME/conf/tomcat-users.xml
```

In Tomcat 9.0 access to the manager application is split between different users. [Read more...](#)

Release Notes

Documentation

[Tomcat 9.0 Documentation](#)
[Tomcat 9.0 Configuration](#)
[Tomcat Wiki](#)

Find additional important configuration information in:

```
$CATALINA_HOME/RUNNING.txt
```

Getting Help

FAQ and Mailing Lists

The following mailing lists are available:

- [tomcat-announce](#)
Important announcements, releases, security vulnerability notifications. (Low volume).
- [tomcat-users](#)
User support and discussion
- [tomcat-dev](#)
Developer discussion

STEP-6: Deploy the application on Tomcat Server

connect the tomcat server and run the script to setup tomcat

Run the below script to setup tomcat

Copy

Copy

```
yum install java-17-amazon-corretto -y
```

```
wget https://dlcdn.apache.org/tomcat/tomcat-9/v9.0.98/bin/apache-tomcat-9.0.98.tar.gz
```

```
tar -zxvf apache-tomcat-9.0.98.tar.gz
```

```
sed -i '56 a<role rolename="manager-gui"/>' apache-tomcat-9.0.98/conf/tomcat-users.xml
```

```
sed -i '57 a<role rolename="manager-script"/>' apache-tomcat-9.0.98/conf/tomcat-users.xml
```

```
sed -i '58 a<user username="tomcat" password="admin@123" roles="manager-gui, manager-script"/>' apache-tomcat-9.0.98/conf/tomcat-users.xml
```

```
sed -i '59 a</tomcat-users>' apache-tomcat-9.0.98/conf/tomcat-users.xml
```

```
sed -i '56d' apache-tomcat-9.0.98/conf/tomcat-users.xml
```

```
sed -i '21d' apache-tomcat-9.0.98/webapps/manager/META-INF/context.xml
```

```
sed -i '22d' apache-tomcat-9.0.98/webapps/manager/META-INF/context.xml
```

```
sh apache-tomcat-9.0.98/bin/startup.sh
```

Remember the tomcat credentials

- username**:** tomcat
- password**:** admin@123

If you wish to modify, change the credentials on above script also.

Now access the tomcat dashboard with **public-ip** of tomcat server with **8080** port.




Tomcat Web Application Manager

Message: OK

Manager

[List Applications](#)
[HTML Manager Help](#)
[Manager Help](#)
[Server Status](#)

Applications					
Path	Version	Display Name	Running	Sessions	Commands
/	None specified	Welcome to Tomcat	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/docs	None specified	Tomcat Documentation	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/examples	None specified	Servlet and JSP Examples	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/host-manager	None specified	Tomcat Host Manager Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/manager	None specified	Tomcat Manager Application	true	1	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes

Now go to **Manager App** and enter the credentials, You will get the **Tomcat Web Application Manager dashboard**.

Dashboard > MyDeployment > Configuration

Configure

- General
- Source Code Management
- Build Triggers
- Build Environment
- Build Steps
- Post-build Actions

Post-build Actions

Deploy war/ear to a container

WAR/EAR files ?

target/*.war

Context path ?

swiggy

Containers

Add Container

☐ Deploy on failure

Add post-build action

Save

Apply

REST API Jenkins 2.479.2

STEP-7: Integrate Tomcat with Jenkins

Configure the Jenkins job and select **Post Build Actions** and select **Deploy war/ear to a container**

WAR/EAR file: **target/*.war**

context path: **swiggy**

The screenshot shows the Jenkins configuration interface for a job named 'MyDeployment'. The breadcrumb navigation at the top indicates the path: Dashboard > MyDeployment > Configuration. On the left, a 'Configure' sidebar lists various settings: General, Source Code Management, Build Triggers, Build Environment, Build Steps, and Post-build Actions (which is currently selected). The main configuration area is titled 'Containers' and contains a single container entry named 'Tomcat 9.x Remote'. This entry has a 'Credentials' dropdown menu set to '- none -', an '+ Add' button, and a 'Tomcat URL' field with a help icon. Below these fields is an 'Advanced' dropdown. At the bottom of the container list is an 'Add Container' button. A checkbox labeled 'Deploy on failure' is also present. Below the container list is an 'Add post-build action' dropdown. At the very bottom of the configuration area are two buttons: 'Save' and 'Apply'.

and click on **add container** and select **Tomcat 9.x Remote**

Jenkins Credentials Provider: Jenkins

Kind

Username with password

Scope ?

Global (Jenkins, nodes, items, all child items, etc)

Username ?

tomcat

☐ Treat username as secret ?

Password ?

.....

ID ?

appserver

Description ?

tomcat credentials

Save Apply

Now add the tomcat credentials, click on **Add** select **Jenkins**

Dashboard > MyDeployment > Configuration

Configure

- General
- Source Code Management
- Build Triggers
- Build Environment
- Build Steps
- Post-build Actions

Containers

Tomcat 9.x Remote

Credentials

tomcat/***** (tomcat credentials)

+ Add

Tomcat URL ?

http://54.88.54.240:8080

Advanced

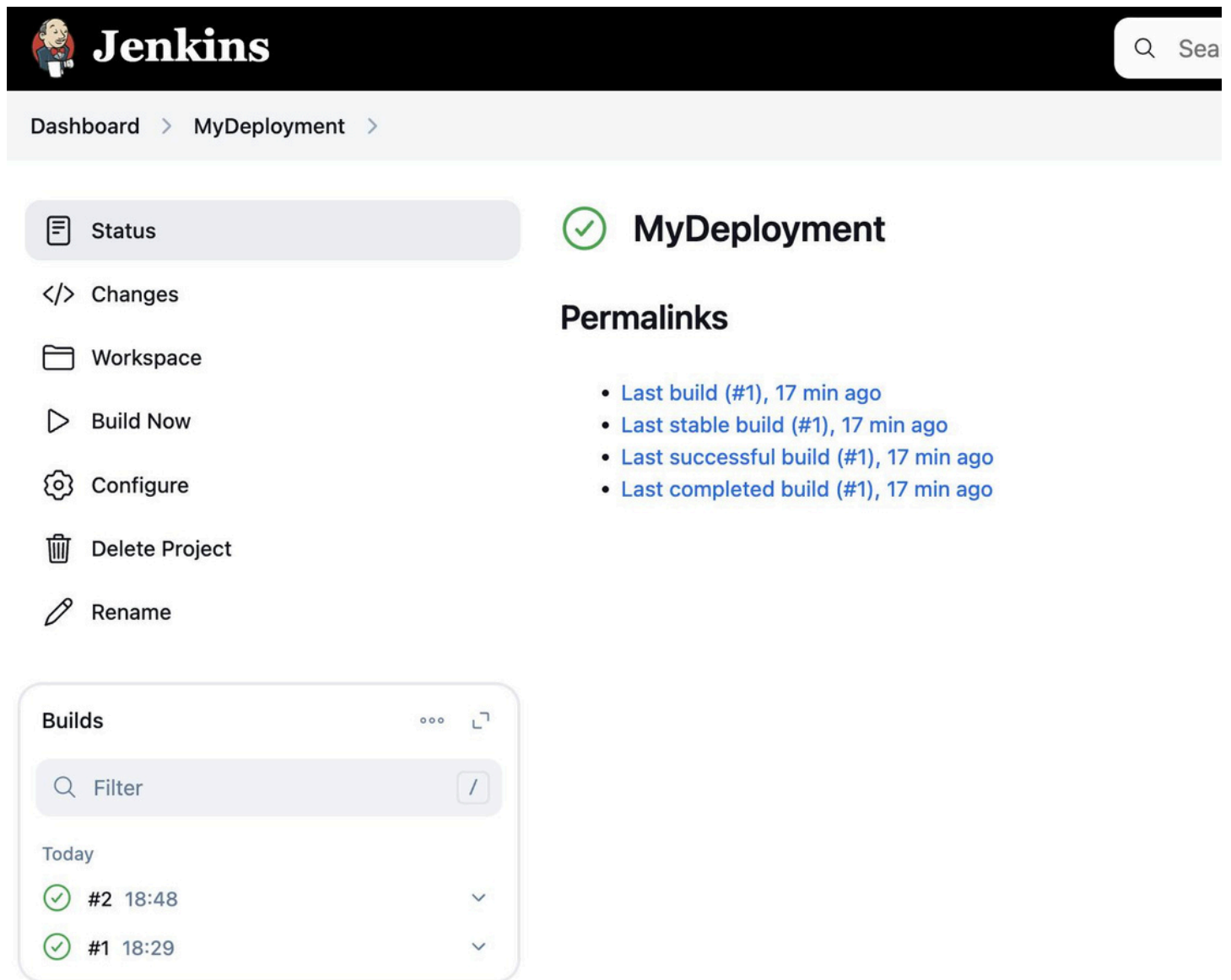
Add Container

☐ Deploy on failure

Add post-build action

Save Apply

Now click on **Add** and select the **credentials** and enter the **tomcat url**



The screenshot shows the Jenkins web interface. At the top is the Jenkins logo and a search bar. Below the logo is a breadcrumb trail: Dashboard > MyDeployment >. On the left side, there is a sidebar with several menu items: Status (highlighted), Changes, Workspace, Build Now, Configure, Delete Project, and Rename. The main content area is titled 'MyDeployment' with a green checkmark icon. Below the title is a section for 'Permalinks' containing four links: 'Last build (#1), 17 min ago', 'Last stable build (#1), 17 min ago', 'Last successful build (#1), 17 min ago', and 'Last completed build (#1), 17 min ago'. At the bottom left, there is a 'Builds' section with a search filter and a list of builds. The list shows two builds: '#2 18:48' and '#1 18:29', both with green checkmarks indicating success.

Jenkins

Dashboard > MyDeployment >

Status

Changes

Workspace

Build Now

Configure

Delete Project

Rename

MyDeployment

Permalinks

- [Last build \(#1\), 17 min ago](#)
- [Last stable build \(#1\), 17 min ago](#)
- [Last successful build \(#1\), 17 min ago](#)
- [Last completed build \(#1\), 17 min ago](#)

Builds

Filter

Today

- ✓ #2 18:48
- ✓ #1 18:29

Now save the job and click on **Build Now**, If the build gets success then our application will be deployed on tomcat successfully.

← → ↻ ⚠ Not Secure 54.88.54.240:8080/manager/html ☆ 👤 ⋮



Tomcat Web Application Manager

Message: OK

Manager

[List Applications](#) [HTML Manager Help](#) [Manager Help](#) [Server Status](#)

Applications

Path	Version	Display Name	Running	Sessions	Commands
/	None specified	Welcome to Tomcat	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/docs	None specified	Tomcat Documentation	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/examples	None specified	Servlet and JSP Examples	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/host-manager	None specified	Tomcat Host Manager Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/manager	None specified	Tomcat Manager Application	true	1	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/swiggy	None specified	Archetype Created Web Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes

The build is **success**, Now lets **refresh** the **tomcat** page and we will see **swiggy** app is **running**

← → ↻ ⚠ Not Secure 54.88.54.240:8080/swiggy/?#projects ☆ 👤 ⋮

Mustafa - DevOps Engineer

Building Automation & Scalable Solutions for Modern Applications

[About](#) [Skills](#) [Projects](#) [Contact](#)

About Me

Hello! I'm Mustafa, a DevOps Engineer passionate about optimizing workflows and delivering scalable, automated solutions. I specialize in tools like Jenkins, Docker, Kubernetes, Terraform, and cloud platforms such as AWS and Azure.

Skills

CI/CD - Jenkins, GitHub Actions Containerization - Docker Orchestration - Kubernetes Infrastructure as Code - Terraform

Cloud Platforms - AWS, Azure Monitoring - Prometheus, Grafana

Open the swiggy app and check the output.

To automate the delivery process, we can also implement Webhooks!.

Best Practices

- Use a separate staging environment for testing deployments before pushing to production.
- Automate testing as part of the CI/CD pipeline to catch bugs early.
- Secure your Jenkins and Tomcat configurations to prevent unauthorized access.

Conclusion

By automating deployment with CI/CD, you save time, reduce errors, and deliver features faster. Deploying Java applications to Tomcat with tools like Jenkins and Maven is straightforward when done step-by-step. Start building your pipeline today to experience the power of automation.